

république Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
Université des Sciences et de la Technologie Houari Boumediene

Faculté d'Informatique
Département SIQ

Mémoire de Master

Filière : Informatique
Spécialité : Réseau Et Systèmes Distribués

Thème:

**Conception et déploiement d'une infrastructure
Cloud privée hyperconvergée basée sur des
technologies open source**

Sujet Proposé par :

Mr : NEFFAH Mohamed

Soutenu le : /.././2026

Devant le jury composé de :

-
-

Présenté par :

- **Bouzara Zakaria**
- **Ouacherine Ilyes**

Binôme N° : 018RSD / 26

Année Universitaire : 2025/2026

I. SOMMAIRE

Introduction Générale.....	7
Chapitre 1 : Présentation du Contexte.....	9
1.1. Introduction.....	9
1.2. Présentation de Sonatrach.....	9
1.3.1. Infrastructure actuelle de Sonatrach.....	10
1.3.2. Les besoins en infrastructure légère pour les zones de test.....	10
1.4. Problématique.....	11
1.5. Objectifs du Projet.....	11
Objectif général.....	11
Objectifs spécifiques.....	11
1.6. Périmètre et Limitations du Projet.....	12
1.7. Conclusion.....	13
Chapitre 2 : État de l'Art et Concepts Théoriques.....	14
2.1. Introduction.....	14
2.2. L'Hyperconvergence (HCI).....	14
2.2.1. Définition.....	14
2.2.2. Avantages de l'hyperconvergence.....	15
2.2.3. HCI vs Infrastructure Traditionnelle.....	15
Tableau 1-1: comparaison du HCI avec Infrastructure Traditionnelle.....	15
2.3. Comparaison : Solutions Propriétaires vs Open Source.....	16
2.3.1. Solutions propriétaires.....	16
2.3.2. Solutions open source.....	16
2.4. Analyse des Solutions Open Source HCI.....	16
2.4.1. OpenStack.....	17
2.4.2. Harvester HCI.....	17
2.4.3. Proxmox VE.....	18
2.4.4. Tableau comparatif.....	18
Tableau 2-1: OpenStack vs Harvester vs Proxmox VE.....	19
2.5. Présentation des Technologies Retenues.....	19
2.5.1. Proxmox VE : L'hyperviseur.....	19
2.5.2. Ceph : Le stockage distribué.....	19
2.5.3. Open vSwitch (OVS) : La virtualisation réseau.....	20
2.6. L'Architecture "Inception" : La Virtualisation Imbriquée.....	21
2.7. Conclusion.....	22

II. Table De Figures

III. Liste Des Tableaux

Tableau 2-1: comparaison du HCI avec Infrastructure Traditionnelle	16
Tableau 2-2: OpenStack vs Harvester vs Proxmox VE	19

IV. Liste Des Abréviations

Abréviation	Signification
BSD	Berkeley Software Distribution
CPU	Central Processing Unit
HA	High Availability
HCI	Hyper Converged Infrastructure
KVM	Kernel-based Virtual Machine
LXC	Linux Containers
MDS	MetaData Server
MGR	Manager
MON	Monitor
OSD	Object Storage Daemon
OVS	Open Virtual Switch
PC	Personal computer
PVE	Proxmox Virtual Environment
RAM	Random Access Memory
RBD	RADOS Block Device
RSD	Réseau Et Systèmes Distribués
TCO	Total Cost of Ownership
vCPU	Virtual Central Processing Unit
VE	Virtual Environment
VM	Virtual Machine

Introduction Générale

La transformation numérique des grandes organisations constitue aujourd'hui un impératif stratégique incontournable. Les entreprises industrielles et les institutions publiques font face à une croissance exponentielle de leurs besoins en ressources informatiques : puissance de calcul, capacité de stockage, disponibilité des services et agilité dans le déploiement des applications. Face à ces exigences croissantes, les architectures informatiques traditionnelles, fondées sur des composants matériels distincts et des solutions propriétaires coûteuses, montrent progressivement leurs limites en termes de flexibilité, de coût et de dépendance technologique.

C'est dans ce contexte que s'imposent les infrastructures Cloud privées hyperconvergées, capables de répondre aux besoins des grandes organisations tout en conservant la maîtrise totale de leurs données et de leurs systèmes. L'hyperconvergence (HCI) représente une rupture architecturale majeure : elle unifie au sein d'une même plateforme logicielle les ressources de calcul, de stockage et de réseau, jusqu'alors gérées séparément par des équipements et des équipes distincts.

Sonatrach, entreprise nationale algérienne et premier groupe industriel du continent africain, opère une infrastructure informatique d'envergure nationale reposant actuellement sur la solution propriétaire Nutanix. Si cette solution répond efficacement aux besoins de production actuels, elle représente un coût de licence croissant et une dépendance technologique forte vis-à-vis d'un éditeur unique. Dans une perspective à moyen et long terme, la question d'une alternative open source souveraine, capable d'assurer les mêmes garanties de fiabilité et de performance à moindre coût, devient stratégiquement critique pour l'entreprise.

Ce projet s'inscrit dans cette vision. Il ne s'agit pas simplement de concevoir un environnement de test léger, mais de poser les bases d'une infrastructure Cloud privée hyperconvergée open source, robuste et évolutive, susceptible de constituer à terme une alternative crédible à Nutanix à l'échelle de l'entreprise.

Ce projet vise à atteindre les objectifs suivants :

- Concevoir une architecture Cloud privée hyperconvergée basée exclusivement sur des outils open source.
- Déployer un cluster de 3 noeuds Proxmox VE avec stockage distribué Ceph et virtualisation réseau via Open vSwitch.
- Assurer la haute disponibilité des services applicatifs grâce aux mécanismes de basculement automatique (HA).
- Démontrer la fiabilité de la solution à travers un scénario de panne contrôlée sur une application web multicouche.
- Produire une documentation technique reproductible, exploitable comme base pour un déploiement à plus grande échelle.

Ce mémoire est organisé en cinq chapitres :

- **Chapitre 1 - Introduction et Contexte** : présente le cadre général, la problématique et les objectifs du projet.
- **Chapitre 2 - Etat de l'Art** : analyse les solutions existantes (propriétaires et open source) et justifie le choix de la stack retenue.
- **Chapitre 3 - Conception de l'Architecture** : détaille l'architecture logique et physique : cluster Proxmox imbriqué, VLANs OVS, dimensionnement Ceph.
- **Chapitre 4 - Implémentation et Déploiement** : documente pas à pas l'installation, la configuration et le déploiement de l'application de démonstration.
- **Chapitre 5 - Validation et Tests** : présente les résultats du scénario de panne, les métriques de performance et les conclusions du projet.

Chapitre 1 : Présentation du Contexte

1.1. Introduction

Ce chapitre vise à présenter le contexte général dans lequel s'inscrit notre projet de fin d'études. Nous y aborderons les fondements de l'hyper convergence (HCI), les enjeux liés à la virtualisation et à la gestion unifiée des ressources informatiques, ainsi que les besoins spécifiques de l'entreprise Sonatrach. Ce cadre introductif nous permettra de mieux comprendre les choix technologiques effectués et les objectifs fixés pour le déploiement d'une infrastructure Cloud privée hyperconvergée basée sur des solutions open source.

1.2. Présentation de Sonatrach

Sonatrach est une entreprise publique algérienne créée le 31 décembre 1963, peu après l'indépendance de l'Algérie. Elle a été fondée dans le but de gérer, développer et exploiter les ressources hydrocarbures du pays, qui représentent une richesse stratégique pour l'économie nationale. À ses débuts, Sonatrach se consacrait essentiellement au transport et à la commercialisation du pétrole brut extrait en Algérie. [1]

Avec le temps, Sonatrach a évolué pour devenir la plus grande entreprise en Afrique dans le domaine des hydrocarbures. Elle intervient aujourd'hui dans l'ensemble de la chaîne de valeur pétrolière et gazière : exploration, production, transport, transformation et commercialisation. L'entreprise est également active dans la pétrochimie et les énergies renouvelables.

Sonatrach entretient de nombreuses collaborations internationales avec des compagnies pétrolières et gazières étrangères, impliquées dans de grands projets de partenariat via des joint-ventures et accords technologiques avec des entreprises européennes, américaines, asiatiques et africaines. Ces coopérations ont permis un transfert de savoir-faire et de technologie vers l'Algérie.

Le projet est réalisé au sein du Centre d'innovation de Sonatrach. Cette direction joue un rôle stratégique dans l'innovation technologique et l'optimisation des processus internes. Elle dispose d'infrastructures informatiques avancées, notamment des solutions de virtualisation et de stockage haute capacité, destinées à soutenir les travaux de simulation, de test et de pré-production.

1.3. Infrastructure Existante et Contexte Technologique

1.3.1. Infrastructure actuelle de Sonatrach

Sonatrach dispose actuellement d'une infrastructure hyperconvergente basée sur la solution propriétaire Nutanix, déployée à l'échelle nationale sur plusieurs wilayas et utilisée pour les environnements de production critiques. Cette solution est reconnue pour sa stabilité, ses performances et la richesse de ses fonctionnalités, notamment en matière de gestion centralisée, de haute disponibilité et de scalabilité.

Cependant, cette infrastructure présente des contraintes structurelles qui deviennent de plus en plus préoccupantes dans une perspective d'évolution à long terme :

- **Coûts de licences croissants** : les frais de licence Nutanix suivent une trajectoire haussière, représentant une charge financière significative et récurrente pour l'entreprise, indépendamment du niveau d'utilisation réel de la plateforme.
- **Dépendance technologique forte** : le recours exclusif à un éditeur propriétaire unique expose Sonatrach à un risque de vendor lock-in, limitant sa capacité à négocier les conditions contractuelles et à adopter librement de nouvelles technologies.
- **Manque de souveraineté technique** : les évolutions de la plateforme, les mises à jour et les décisions d'architecture sont entièrement soumises à la roadmap de l'éditeur, sans possibilité d'adaptation aux besoins spécifiques de l'entreprise.
- **Inadaptation aux petits déploiements** : pour les environnements de test, de développement et de pré-production, le déploiement de Nutanix sur de petits clusters de 3 à 4 noeuds est disproportionné par rapport aux besoins réels, tant en complexité qu'en coût.

Ces constats positionnent ce projet non pas comme un simple exercice académique, mais comme une réponse concrète à une problématique stratégique réelle : explorer et valider la viabilité d'une infrastructure hyperconvergée open source capable de répondre aux exigences de Sonatrach, aujourd'hui en environnement de test et demain potentiellement en production.

1.3.2. Les besoins en infrastructure légère pour les zones de test

Au-delà des besoins immédiats en environnements de test et de pré-production, Sonatrach a besoin d'une infrastructure qui réponde à des exigences de niveau entreprise, tout en s'affranchissant des contraintes des solutions propriétaires. Ces besoins couvrent plusieurs dimensions :

- **Fiabilité et haute disponibilité** : l'infrastructure doit garantir la continuité de service en cas de défaillance d'un ou plusieurs noeuds, avec basculement automatique des charges de travail sans intervention humaine.
- **Performance et scalabilité** : la solution doit pouvoir monter en charge progressivement, en ajoutant des noeuds au cluster sans interruption de service, pour accompagner la croissance des besoins de l'entreprise.
- **Gestion unifiée des ressources** : calcul, stockage et réseau doivent être administrés depuis une interface centralisée unique, réduisant la complexité opérationnelle et les besoins en compétences spécialisées multiples.
- **Équilibrage dynamique de charge** : à l'image du mécanisme DRS de VMware, la plateforme doit être capable de redistribuer automatiquement les charges de travail entre les noeuds en fonction de l'utilisation des ressources, optimisant ainsi l'efficacité globale du cluster.
- **Souveraineté et indépendance technologique** : en s'appuyant exclusivement sur des technologies open source, l'entreprise reprend le contrôle total de son infrastructure, avec la liberté d'auditer, de modifier et d'adapter la solution à ses besoins propres.
- **Réduction du coût total de possession (TCO)** : l'absence de licences propriétaires représente une économie substantielle sur le long terme, réallouable vers des investissements matériels ou humains.

1.4. Problématique

La problématique centrale de ce projet est la suivante :

Comment concevoir et déployer une infrastructure Cloud privée hyperconvergée, entièrement basée sur des technologies open source, capable de répondre aux exigences de fiabilité, de performance et de scalabilité d'une grande entreprise industrielle, tout en constituant une alternative crédible et souveraine aux solutions propriétaires de type Nutanix ?

Cette problématique soulève plusieurs questions techniques structurantes :

- Comment concevoir un système de stockage distribué, redondant et performant garantissant l'intégrité des données en cas de défaillance d'un nœud ?
- Comment assurer le basculement et l'équilibrage automatique des charges de travail dans un cluster à plusieurs nœuds, sans perte de données ni interruption prolongée de service ?
- Comment segmenter et isoler strictement les flux réseau au sein d'une infrastructure hyperconvergée, en séparant les trafics de gestion, de réplication, de migration et de production ?
- Dans quelle mesure une solution open source peut-elle répondre aux exigences d'une infrastructure de niveau production dans le contexte d'une grande entreprise ?

1.5. Objectifs du Projet

Objectif général

Concevoir et déployer une infrastructure Cloud privée hyperconvergée, entièrement open source, offrant des garanties de fiabilité, de performance et d'évolutivité comparables aux solutions propriétaires de type Nutanix, et susceptible de constituer une alternative stratégique pour Sonatrach à moyen et long terme.

Objectifs spécifiques

- Déployer un cluster de trois nœuds à haute disponibilité, avec gestion centralisée, administration simplifiée et support de la migration à chaud.
- Concevoir et mettre en place un système de stockage distribué, redondant et performant intégrant les composants physiques et logiques nécessaires à la cohérence des données.
- Implémenter une segmentation réseau avancée, en configurant une architecture multi-VLAN séparant les flux de gestion, de réplication, de migration et de production.
- Développer un mécanisme d'équilibrage dynamique de la charge, capable de re-distribuer automatiquement les machines virtuelles selon les métriques de performance en temps réel.
- Valider la solution à travers un scénario de haute disponibilité démontrant la tolérance aux pannes sans perte de données ni interruption de service.
- Fournir une base d'infrastructure évolutive, accompagnée d'une documentation technique et d'une conception architecturale réutilisable à plus grande échelle.

1.6. Périmètre et Limitations du Projet

Ce projet vise le déploiement d'un cluster hyperconvergé open source destiné aux environnements de test, de développement et de pré-production. L'infrastructure est déployée sur un serveur dédié mis à disposition par Sonatrach, offrant les ressources matérielles nécessaires à l'exécution du cluster Proxmox VE, du stockage distribué Ceph et des VMs de démonstration.

Le périmètre de ce projet se limite volontairement à un environnement de laboratoire structuré, ce qui implique les contraintes suivantes :

- **Architecture sur serveur unique** : les trois nœuds Proxmox étant hébergés sur une même machine physique via virtualisation imbriquée, une panne matérielle totale du serveur hôte affecterait l'ensemble du cluster simultanément. Ce scénario ne se produirait pas dans un déploiement physique distribué sur plusieurs serveurs.
- **Absence de multitenancy** : Proxmox VE dans sa configuration actuelle est orienté cloud privé. La gestion multi-tenant propre aux clouds publics n'entre pas dans le périmètre de ce projet.
- **Capacité de stockage limitée** : la taille du pool Ceph est contrainte par l'espace disque disponible sur le serveur, ce qui rend cette infrastructure adaptée aux charges de test mais pas aux workloads de données à grande échelle.
- **Maintenance à long terme** : la gestion continue de l'infrastructure (mises à jour, sauvegardes automatisées, gestion des pannes matérielles) n'est pas couverte par ce projet et constitue une perspective d'évolution naturelle.

1.7. Conclusion

Dans ce chapitre, nous avons présenté le contexte général du projet, les besoins identifiés au sein de Sonatrach, et la problématique qui motive notre démarche. Nous avons défini les objectifs, le périmètre et les limites du projet. Ce cadre nous permettra d'aborder, dans le chapitre suivant, l'état de l'art des solutions disponibles et les fondements théoriques de l'hyperconvergence, de la virtualisation et des technologies retenues.

Chapitre 2 : État de l'Art et Concepts Théoriques

2.1. Introduction

Dans un contexte de transformation numérique accélérée, les organisations scientifiques et industrielles cherchent des solutions d'infrastructure à la fois performantes, flexibles et économiques. Ce chapitre présente les concepts théoriques fondamentaux sur lesquels repose notre projet : l'hyperconvergence, la virtualisation, le stockage distribué et la virtualisation du réseau. Il dresse également un état de l'art comparatif des solutions disponibles, afin de justifier les choix technologiques retenus.

2.2. L'Hyperconvergence (HCI)

2.2.1. Définition

L'hyperconvergence (HCI, Hyper-Converged Infrastructure) est une architecture informatique qui regroupe, dans un même système, les ressources de calcul, de stockage et de réseau. Ces ressources sont pilotées par une couche logicielle centralisée, supprimant ainsi la nécessité de gérer des équipements spécialisés distincts pour chaque fonction. [2]

L'infrastructure hyperconvergée repose sur trois composantes fondamentales :

- **Le calcul** : des processeurs virtuels (vCPU) alloués dynamiquement aux machines virtuelles.
- **Le stockage** : un système de stockage distribué logiciel, agrégé à partir des disques locaux de chaque nœud.
- **Le réseau** : un switch virtuel logiciel permettant la segmentation et l'isolation du trafic réseau.

2.2.2. Avantages de l'hyperconvergence

L'approche HCI présente plusieurs avantages par rapport aux architectures traditionnelles à trois niveaux (compute, storage, network séparés) :

- **Simplicité d'administration** : une interface unique pour gérer l'ensemble des ressources.
- **Réduction des coûts matériels** : pas besoin de baies de stockage ou de switchs réseau dédiés.
- **Évolutivité horizontale** : l'ajout d'un nœud augmente simultanément la capacité de calcul, de stockage et de réseau.
- **Haute disponibilité native** : la réplication des données entre les nœuds assure la continuité de service en cas de panne.

2.2.3. HCI vs Infrastructure Traditionnelle

Critère	Infrastructure Traditionnelle	Infrastructure HCI
Gestion	Silos séparés (compute, storage, network)	Interface unifiée
Coût initial	Elevé (équipements spécialisés)	Réduit (commodity hardware)
Évolutivité	Verticale (upgrade de matériel)	Horizontale (ajout de nœuds)
Complexité	Haute	Faible
Haute disponibilité	Optionnelle et complexe	Native

Tableau 2-1: comparaison du HCI avec Infrastructure Traditionnelle

2.3. Comparaison : Solutions Propriétaires vs Open Source

2.3.1. Solutions propriétaires

Les solutions propriétaires telles que **VMware vSAN** ou **Nutanix** offrent un support professionnel, des fonctionnalités avancées et une interface utilisateur soignée. Cependant, elles présentent plusieurs limites importantes dans notre contexte :

- **Coûts élevés** : les licences et la maintenance annuelle représentent un investissement significatif, difficilement justifiable pour des environnements de test.
- **Dépendance fournisseur (Vendor Lock-in)** : la migration vers une autre solution est souvent complexe et coûteuse.
- **Faible flexibilité** : les environnements propriétaires offrent peu de liberté pour personnaliser ou adapter l'infrastructure.

2.3.2. Solutions open source

Les solutions open source présentent plusieurs avantages stratégiques :

- **Indépendance technologique** : aucune dépendance vis-à-vis d'un éditeur.
- **Absence de coûts de licence** : seuls les coûts d'infrastructure matérielle et de main-d'œuvre sont à prévoir.
- **Transparence et auditabilité** : le code source est accessible et auditable.
- **Communauté active** : documentation abondante et mises à jour régulières.

Dans un contexte de zones de test et de petits déploiements, l'approche open source est clairement la plus adaptée.

2.4. Analyse des Solutions Open Source HCI

Le critère principal de sélection est la légèreté et la capacité à fonctionner de manière stable sur des clusters de 3 à 4 nœuds.

2.4.1. OpenStack

OpenStack est une plateforme cloud open source très complète et standardisée, utilisée par de nombreuses grandes entreprises et fournisseurs cloud. Elle propose une large gamme de services : calcul (Nova), stockage objet (Swift), stockage en bloc (Cinder), réseau (Neutron), identité (Keystone), etc.

Cependant, OpenStack présente des inconvénients majeurs pour notre cas d'usage :

- **Complexité de déploiement** : l'installation et la configuration d'OpenStack nécessitent une expertise poussée et prennent un temps considérable.
- **Consommation de ressources élevée** : les services de base d'OpenStack consomment à eux seuls plusieurs Go de RAM, avant même de démarrer des VMs de production.
- **Inadapté aux petits clusters** : OpenStack est conçu pour des déploiements de grande envergure. Sur 3 nœuds, ses performances et sa stabilité sont compromises.

2.4.2. Harvester HCI

Harvester est une solution HCI open source développée par SUSE, basée sur Kubernetes et KubeVirt. Elle propose une interface moderne et une intégration native avec Rancher pour la gestion de clusters Kubernetes. [5]

Ses limites dans notre contexte :

- **Consommation en RAM et CPU élevée** : Harvester nécessite des ressources importantes pour faire fonctionner la couche Kubernetes sous-jacente, avant même d'allouer des ressources aux VMs.

- **Maturité relative pour les VMs classiques** : Harvester est davantage orienté vers les workloads conteneurisés. La gestion de VMs Linux/Windows classiques est moins optimisée que sur une solution dédiée.
- **Courbe d'apprentissage** : la maîtrise de Kubernetes est un prérequis pour administrer efficacement Harvester. [4]

2.4.3. Proxmox VE

Proxmox Virtual Environment (VE) est une plateforme de virtualisation open source basée sur KVM (pour les VMs) et LXC (pour les conteneurs). Elle propose une interface web centralisée, un support natif de Ceph pour le stockage distribué, et une gestion intégrée du clustering et de la haute disponibilité. [6]

Ses atouts pour notre cas d'usage sont nombreux :

- **Légèreté** : moins de 1 Go de RAM consommé à vide sur le nœud, laissant l'essentiel des ressources aux VMs.
- **Déploiement rapide** : un cluster de 3 nœuds est opérationnel en quelques heures.
- **Interface web intuitive** : la console Proxmox (accessible sur le port 8006) centralise la gestion des VMs, conteneurs, stockage, réseau et haute disponibilité.
- **Intégration native de Ceph** : Proxmox intègre nativement les outils de gestion Ceph, simplifiant considérablement le déploiement du stockage distribué.
- **Communauté active** : documentation officielle complète et forums très actifs.

2.4.4. Tableau comparatif

Critère	OpenStack	Harvester	Proxmox VE
Légèreté	Faible	Moyenne	Elevée
Facilité de déploiement	Difficile	Moyenne	Facile
Adapté aux petits clusters (3 noeuds)	Non	Partiel	Oui
Intégration stockage distribué	Via Cinder/Ceph	Via Longhorn	Ceph natif
Gestion des VMs classiques	Bonne	Moyenne	Excellente
Maturité	Très mature	En cours	Mature
Coût de licence	Gratuit	Gratuit	Gratuit (Enterprise payant)

Tableau 2-2: OpenStack vs Harvester vs Proxmox VE

Conclusion : Proxmox VE est la solution la mieux adaptée à nos contraintes. Elle sera retenue comme hyperviseur principal du projet.

2.5. Présentation des Technologies Retenues

2.5.1. Proxmox VE : L'hyperviseur

Proxmox VE est une solution de virtualisation complète basée sur un noyau Linux Debian modifié. Elle supporte deux types de virtualisation :

- **KVM (Kernel-based Virtual Machine)** : pour les machines virtuelles complètes, compatibles avec tout système d'exploitation (Linux, Windows, BSD...).
- **LXC (Linux Containers)** : pour les conteneurs Linux légers, partageant le noyau de l'hôte.

La gestion du cluster Proxmox repose sur **Corosync**, un framework de communication de cluster qui assure le consensus entre les nœuds (quorum). Un cluster Proxmox nécessite un minimum de 3 nœuds pour garantir un quorum stable et éviter les situations de “split-brain”.

2.5.2. Ceph : Le stockage distribué

Ceph est une solution libre de stockage distribuée (Software-Defined Storage) qui s'impose aujourd'hui comme une référence dans le domaine des infrastructures open source. Conçu pour répondre aux besoins de fiabilité, d'extensibilité et de performance, Ceph constitue le choix idéal pour assurer la couche de stockage d'une infrastructure hyperconvergée telle que celle que nous proposons. Sa capacité à fonctionner sur du matériel standard (serveurs x86) et son architecture auto-réparatrice en font un composant central de notre projet.

Architecture de Ceph

L'architecture de Ceph repose sur un composant fondamental appelé RADOS (Reliable Autonomic Distributed Object Store). RADOS constitue le cœur du système, chargé de la gestion, de la réplication et de la répartition des données sur l'ensemble du cluster. Autour de ce cœur gravitent plusieurs types de démons qui interagissent pour former un cluster cohérent et performant :

- Ceph Monitor (MON) : Les Moniteurs sont les gardiens du cluster. Ils maintiennent une carte maîtresse de l'état du cluster (cluster map), contenant la liste des nœuds, des OSD, et les règles de distribution des données. Pour assurer une haute disponibilité et éviter les conflits, un cluster doit comporter au minimum trois Moniteurs, en nombre impair, permettant une prise de décision par majorité en cas de panne.
- Ceph Manager (MGR) : Le Manager agit comme un chef d'orchestre. Il collecte toutes les métriques et informations d'état du cluster en temps réel (utilisation du stockage, performances, statistiques). Il expose ces données via une interface web de monitoring et les met à disposition des systèmes de gestion externes, facilitant ainsi l'administration et la supervision.

- Ceph OSD (Object Storage Daemon) : L'OSD est le démon qui gère les périphériques de stockage physiques ou logiques (disques, partitions, volumes logiques). Chaque OSD est responsable de la lecture et de l'écriture des données sur son disque. Les OSDs communiquent entre eux pour assurer la réplication, le rééquilibrage et la réparation des données en cas de panne. Dans une architecture hyperconvergente, les OSDs résident sur les mêmes nœuds que l'hyperviseur (Proxmox), chaque nœud contribuant ainsi au stockage global.

- Ceph Metadata Server (MDS) : Ce démon est spécifique au système de fichiers distribué CephFS. Il gère les métadonnées des fichiers (noms, permissions, arborescences). Pour notre projet, qui utilise principalement le stockage en mode bloc pour les machines virtuelles, ce composant n'est pas requis.

La gestion intelligente des données : CRUSH et Placement Groups

L'intelligence de Ceph réside dans sa manière unique de gérer l'emplacement des données. Contrairement aux systèmes de stockage traditionnels qui utilisent une table de correspondance centralisée, Ceph utilise un algorithme appelé CRUSH (Controlled Replication Under Scalable Hashing). Cet algorithme permet à un client de calculer mathématiquement l'emplacement exact d'un objet (ou de sa réplique) en se basant sur la carte CRUSH, sans avoir à interroger un serveur central. Cette approche élimine un point de défaillance unique et améliore considérablement la scalabilité.

La carte CRUSH est une structure hiérarchique qui définit l'organisation physique du cluster. L'administrateur peut y définir des "domaines de défaillance" (nœuds, racks, salles, data centers). Ceph utilise ensuite ces informations pour placer les répliques des données sur des nœuds distincts, garantissant ainsi que la perte d'un équipement n'entraîne pas de perte de données.

Pour optimiser la gestion des milliers d'objets, Ceph les regroupe dans des unités logiques appelées Placement Groups (PGs). Les PGs constituent une abstraction intermédiaire entre l'objet et l'OSD, permettant un équilibrage efficace des données et une meilleure performance du cluster.

Les modes de protection des données

Pour assurer la fiabilité et la disponibilité des données, Ceph propose deux mécanismes principaux, adaptés à différents besoins :

- Réplication : C'est le mode le plus simple et le plus performant. Chaque objet est dupliqué en N copies (généralement 3) et chaque copie est placée sur des OSDs différents, dans des

domaines de défaillance distincts. En cas de panne d'un OSD ou d'un nœud, les données restent disponibles sur les autres répliques. Ce mode offre une reconstruction rapide et est idéal pour les charges de travail exigeant de hautes performances en écriture, comme les disques de machines virtuelles. Dans notre projet, nous utiliserons une réplication de facteur 3 pour garantir une haute disponibilité sur le cluster.

- Erasure Coding (Codage d'effacement) : Ce mode est plus efficace en termes d'espace de stockage. Les données sont découpées en K fragments de données, puis encodées avec M fragments de parité, pour un total de K+M fragments répartis sur différents OSDs. Ce mode permet de tolérer la perte de *M* fragments tout en reconstruisant les données initiales. Il est souvent utilisé pour les données froides ou l'archivage, car il nécessite moins d'espace que la réplication (ex: 1,5x l'espace brut contre 3x pour la réplication), mais il est plus coûteux en calcul lors des opérations de reconstruction.

Intégration de Ceph dans une architecture hyperconvergée

L'intégration de Ceph avec Proxmox VE, notre hyperviseur choisi, est native et profondément intégrée. Proxmox offre une interface unifiée qui permet de créer et de gérer un cluster Ceph directement depuis son interface web, simplifiant considérablement les opérations de déploiement et de maintenance.

Dans notre architecture, chaque nœud Proxmox joue un double rôle :

- Rôle de calcul : Il exécute les machines virtuelles via KVM.
- Rôle de stockage : Il contribue au stockage global via ses OSDs Ceph.

Cette convergence des rôles est l'essence même de l'hyperconvergence. Les volumes de stockage des VMs sont fournis via Ceph RBD (RADOS Block Device) , qui expose des périphériques de stockage en bloc sur le réseau. Ces volumes sont distribués et répliqués sur l'ensemble du cluster, assurant ainsi la persistance des données même en cas de panne d'un nœud.

Les avantages apportés par Ceph dans notre infrastructure sont multiples :

Caractéristique	Avantage pour l'infrastructure hyperconvergée
Évolutivité horizontale	L'ajout d'un nouveau nœud apporte simultanément de la puissance de calcul et de la capacité de stockage. Ceph rééquilibre

	automatiquement les données, permettant une croissance linéaire et sans interruption.
Haute disponibilité	L'absence de point de défaillance unique (SPOF) et la réplication des données sur plusieurs nœuds garantissent que le système de stockage reste opérationnel même en cas de panne d'un ou plusieurs serveurs.
Auto-réparation	En cas de panne d'un disque ou d'un nœud, le cluster détecte l'incident et commence immédiatement à recréer les répliques manquantes sur les autres nœuds, assurant ainsi la pérennité des données sans intervention manuelle.
Coût maîtrisé	Ceph est open source et fonctionne sur du matériel standard (x86), ce qui le rend bien moins coûteux que les solutions propriétaires comme les SAN ou les appliances HCI propriétaires.
Simplicité de gestion	L'unification du stockage via Proxmox et les outils modernes comme 'cephadm' (permettant un déploiement conteneurisé et simplifié) réduisent considérablement la complexité administrative.

Conclusion

Ceph représente bien plus qu'un simple système de stockage. Son architecture robuste, sa scalabilité horizontale et sa capacité d'auto-guérison s'alignent parfaitement avec les principes d'une infrastructure hyperconvergée. En choisissant Ceph comme brique de stockage, nous posons des fondations solides pour une infrastructure IT moderne, agile et résiliente, capable de s'adapter aux besoins futurs. L'intégration native avec Proxmox VE permet une gestion centralisée et simplifiée, essentielle pour les environnements de test et de pré-production visés par ce projet.

2.5.3. Open vSwitch (OVS) : La virtualisation réseau

Open vSwitch (OVS) est un commutateur logiciel multicouche open source, distribué sous licence Apache 2.0, dont l'objectif est de fournir une plateforme de commutation de qualité production supportant les interfaces de gestion standard et ouvrant les fonctions de transfert à l'extension programmatique [8]. Particulièrement adapté aux environnements de virtualisation, OVS est conçu pour être distribué sur plusieurs serveurs physiques et supporte les technologies de virtualisation Linux dont KVM et VirtualBox [8]. Il fonctionne principalement en deux composants : **ovs-vswitchd**, le démon qui implémente le switch avec son module noyau Linux pour un transfert haute performance, et **ovsdb-server**, une base de données légère de configuration [8]. Parmi ses fonctionnalités natives figurent le modèle VLAN 802.1Q, le bonding avec ou sans LACP, la supervision via NetFlow et sFlow, la configuration QoS, ainsi que le support des tunnels GRE, VXLAN et Geneve [9].

Dans notre infrastructure, OVS joue le rôle de commutateur central du cluster, assurant la segmentation de tout le trafic en flux distincts et isolés. Cette segmentation est imposée par la coexistence sur la même infrastructure physique de trafics aux profils radicalement différents : réplication Ceph à très haute bande passante, heartbeats Corosync sensibles à la latence, pics de migration à chaud, et workloads applicatifs variables. Sans isolation stricte, ces flux en compétition dégradent mutuellement leurs performances et menacent la stabilité du cluster. OVS résout ce problème depuis un bridge unique par noeud, à l'image de la séparation des réseaux CVM pratiquée par Nutanix.

La conception de notre architecture réseau a été guidée par plusieurs facteurs déterminants.

- Le premier est la nature hétérogène des flux coexistant dans un cluster HCI : chaque type de trafic présente un profil distinct en termes de bande passante, de sensibilité à la latence et de criticité opérationnelle, rendant leur cohabitation sur un réseau commun incompatible avec les exigences de performance et de stabilité d'une infrastructure de niveau entreprise.

- Le deuxième facteur est la référence aux recommandations officielles de la documentation Proxmox et aux guidelines Ceph, qui préconisent explicitement la séparation des réseaux public et cluster Ceph [10] [12], ainsi que l'isolation du trafic Corosync [11].

- Le troisième facteur est le positionnement du projet comme alternative à Nutanix : nous avons analysé la manière dont Nutanix segmente ses propres réseaux internes entre trafic CVM, trafic de réplication et trafic VM, et avons traduit cette logique en VLANs OVS équivalents.

- Enfin, la nécessité d'anticiper la scalabilité du cluster nous a conduits à prévoir dès la conception des segments dédiés à des fonctions comme la migration automatique et la sauvegarde, que le mécanisme DRS que nous développons sollicitera de manière intensive.

Ces paramètres ont abouti à une architecture à neuf VLANs distincts, chacun correspondant à un type de trafic précis avec une plage d'adressage dédiée :

- **VLAN 10 - Management (192.168.10.0/24) :** accès à l'interface web Proxmox, SSH et API. Faible volume mais disponibilité permanente requise.
- **VLAN 20 - Corosync (192.168.20.0/24) :** heartbeats et quorum du cluster.
Extrêmement sensible à la latence : toute perturbation peut déclencher un basculement non souhaité.
- **VLAN 30 - Ceph Cluster Network (10.10.30.0/24) :** communications internes entre OSDs, réplication, rebalancing et recovery. Trafic continu à très haute bande passante, isolé conformément aux recommandations officielles Ceph. [10] [12] [13]
- **VLAN 40 - Ceph Public Network (10.10.40.0/24) :** accès des VMs au stockage Ceph via RBD. Séparé du VLAN 30 pour éviter que les opérations internes Ceph ne dégradent les performances applicatives. [12] [13]
- **VLAN 50 - Live Migration (10.10.50.0/24) :** migrations à chaud entre noeuds. Trafic soutenu et ponctuel, particulièrement sollicité par le mécanisme DRS que nous développons.

- **VLAN 60 - VM Production (172.16.60.0/24)** : workloads applicatifs de production, dont Sona-Web. Point d'entrée utilisateur via HAProxy.
- **VLAN 70 - VM Test/Dev (172.16.70.0/24)** : environnements de développement et validation, strictement isolés de la production.
- **VLAN 80 - DMZ / External (172.16.80.0/24)** : services exposés vers l'extérieur de l'entreprise, protégés par pare-feu. HAProxy est le seul point d'entrée vers cette zone.
- **VLAN 90 - Backup (10.10.90.0/24)** : transferts de sauvegarde et snapshots Proxmox, isolés pour ne pas impacter les workloads en production.

2.6. L'Architecture "Inception" : La Virtualisation

Imbriquée

Pour pallier l'absence temporaire d'un serveur physique dédié, notre projet adopte une architecture de **virtualisation imbriquée** (nested virtualization), parfois appelée architecture "Inception".

Le principe est le suivant :

- **Niveau 0 (Physique)** : un ou deux PCs personnels hébergent Proxmox VE en tant qu'hyperviseur "Master".
- **Niveau 1 (VMs)** : à l'intérieur de cet hyperviseur, on crée 3 VMs de grande taille qui jouent le rôle des nœuds Proxmox du cluster (PVE-01, PVE-02, PVE-03).
- **Niveau 2 (VMs dans VMs)** : le cluster Proxmox de niveau 1 héberge à son tour les VMs de démonstration (Load Balancer, Web Servers, Base de données).

Cette approche présente plusieurs avantages pratiques :

- **Portabilité** : l'ensemble du cluster peut être sauvegardé ou migré facilement.

- **Snapshots avant démo** : il est possible de prendre un snapshot complet du cluster avant une démonstration, permettant de revenir à un état stable en quelques secondes si nécessaire.
- **Réseau 100% virtuel** : les communications entre les 3 nœuds passent par le bus mémoire de l'hôte, éliminant les problèmes de câblage ou de switch physique instable.

Lorsque le serveur Sonatrach sera disponible, la migration sera transparente : les 3 VMs nœuds pourront être exportées et hébergées directement sur le serveur physique, améliorant ainsi les performances globales.

2.7. Conclusion

Ce chapitre a permis de poser les bases théoriques de notre projet. Nous avons défini le concept d'hyperconvergence, comparé les principales solutions open source disponibles et justifié le choix de la stack Proxmox VE + Ceph + Open vSwitch. Nous avons également présenté le rôle de chaque composant dans l'architecture et introduit le concept de virtualisation imbriquée qui sera utilisé dans la phase d'implémentation. Le chapitre suivant présentera la conception détaillée de l'architecture proposée.

Chapitre 3 : Conception

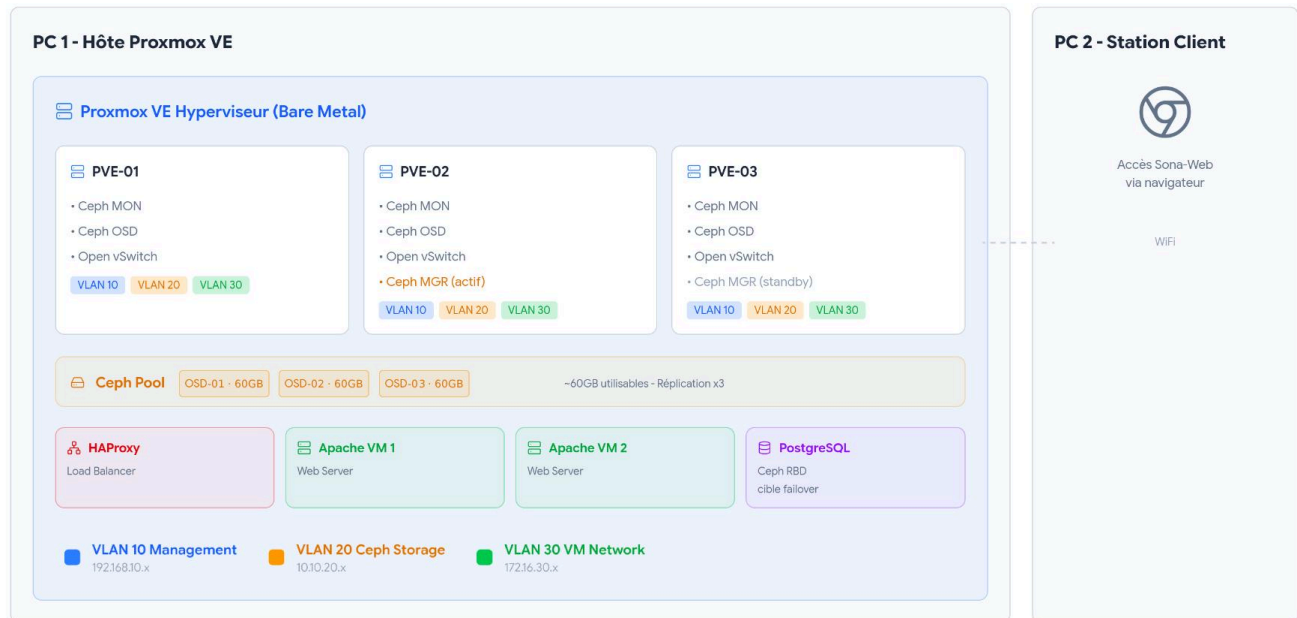


figure 3-1: Architecture de la solution proposé

Bibliographie

- [1] I. N. Sonatrach. (2012, août 13). « *Historique et évolution* ». Institut-numérique. <https://www.institut-numerique.org/2-historique-et-evolution-5028f58cb09d3>
- [2] Schneider, J., Susnjara, S., et Smalley, I. (2024, 12 février). « *Hyperconverged infrastructure explained* ». IBM. <https://www.ibm.com/think/topics/hyperconverged-infrastructure>
- [3] MacAlpine, C. (2025, 27 mai). « *The top 10 hyperconverged infrastructure (HCI) software solutions* ». Expert Insights. <https://expertinsights.com/it-infrastructure/top-hyperconverged-infrastructure-hci-software>
- [4] Pande, A. (2025, February 23). « *Proxmox vs. Harvester: Can the enterprise-grade platform beat the community favorite?* » XDA Developers. <https://www.xda-developers.com/proxmox-vs-harvester/>
- [5] SUSE. (s. d.). « Harvester HCI documentation ». Harvester HCI. <https://harvesterhci.io/>
- [6] Proxmox Server Solutions. (s. d.). « *Proxmox virtual environment overview* ». Proxmox. <https://www.proxmox.com/en/products/proxmox-virtual-environment/overview>
- [7] Sahin, S. (2025, 2 octobre). « *Proxmox NAT configuration* ». ITTechNetworkK. <https://ittechnetworkk.github.io/posts/proxmoxnatconfiguration/proxmox-nat-configuration/>
- [8] Open vSwitch Authors. (s. d.). « *What is Open vSwitch ?* ». Documentation Open vSwitch. <https://docs.openvswitch.org/en/latest/intro/what-is-ovs/>
- [9] Open vSwitch Authors. (s. d.). « *Features* ». Open vSwitch. <https://www.openvswitch.org/features/>
- [10] Proxmox Server Solutions. (2025, 19 novembre). « *Deploy hyper-converged Ceph cluster* ». Proxmox VE Wiki. https://pve.proxmox.com/wiki/Deploy_Hyper-Converged_Ceph_Cluster
- [11] Proxmox Server Solutions. (2025, 18 décembre). « *Full mesh network for Ceph server* ». Proxmox VE Wiki. https://pve.proxmox.com/wiki/Full_Mesh_Network_for_Ceph_Server
- [12] Ceph Authors. (s. d.). « *Network configuration reference* ». Documentation Ceph. <https://docs.ceph.com/en/latest/rados/configuration/network-config-ref/>
- [13] Red Hat. (s. d.). « *Chapter 1 : The Ceph architecture* ». Red Hat Ceph Storage 4 Architecture Guide. https://docs.redhat.com/en/documentation/red_hat_ceph_storage/4/html/architecture_guide/the-ceph-architecture_arch
- [14] Chandak, A., Jaju, K., Kanfode, A., Lohiya, P., et Joshi, A. (2012, mai). « *Dynamic load balancing of virtual machines using QEMU-KVM* ». International Journal of Computer Applications, 46(6), 10-14. <https://research.ijcaonline.org/volume46/number6/pxc3879263.pdf>

- [15] Proxmox Server Solutions. (2025, 6 août). « *Proxmox Backup Server* ». Proxmox Backup Server Wiki. https://pbs.proxmox.com/wiki/Main_Page
- [16] Mr.PlanB. (2025, 9 septembre). « *Goodbye DRS, hello PBS : What I wish I knew before ditching VMware for Proxmox* ». Medium.
<https://medium.com/@PlanB./goodbye-drs-hello-pbs-what-i-wish-i-knew-before-ditching-vmware-for-proxmox-26f9dec3e9f5>
- [17] Watts, J. (1995). « *A practical approach to dynamic load balancing* » [Mémoire de master, California Institute of Technology]. Caltech THESIS.
https://thesis.caltech.edu/6924/1/Watts_j_1995.pdf
- [18] Broadcom. (s. d.). « *VMware DRS overview : Optimizing resource allocation in vSphere cluster* ». Broadcom Knowledge Base.
<https://knowledge.broadcom.com/external/article/391137/vmware-drs-overview-optimizing-resource.html>
- [20]
- [21]
- [22]
- [23]
- [24]
- [25]
- [26]